AIML231 —Techniques in Machine Learning

Week 3

Classification: Decision Trees

Dr Qi Chen

School of Engineering and Computer Science

Victoria University of Wellington

Qi.Chen@vuw.ac.nz

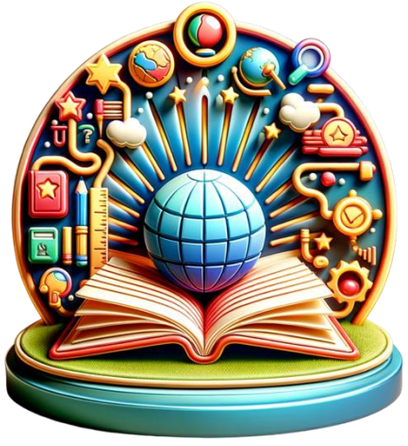
Overview

★ Basic Performance Evaluation Metrics

★ Decision Tree

- ❑ Decision Tree vs. Decision Tree Learning
- ❑ How to build a decision tree using training data
- ❑ How to measure the uncertainty of a DT node:

  Entropy and Information Gain

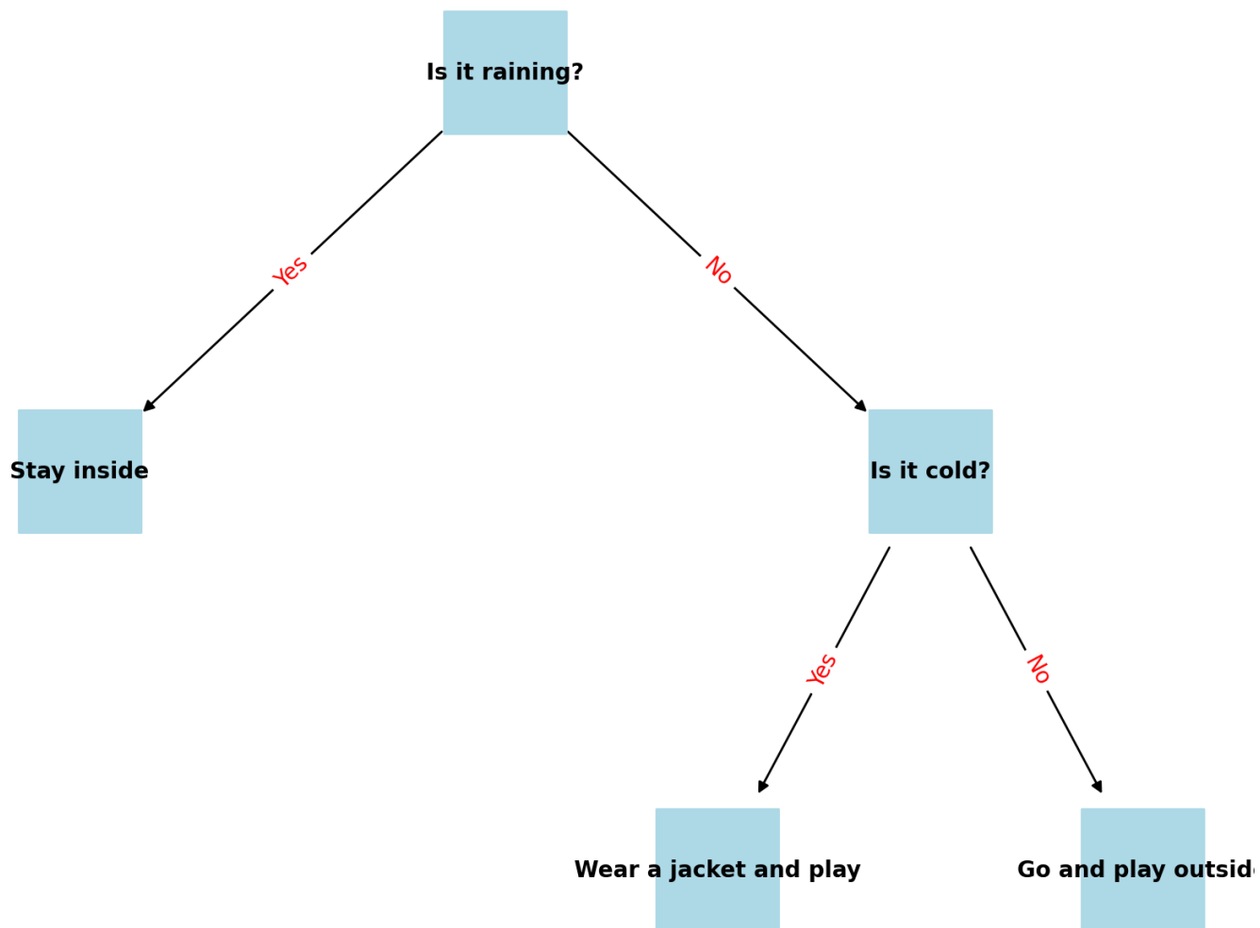


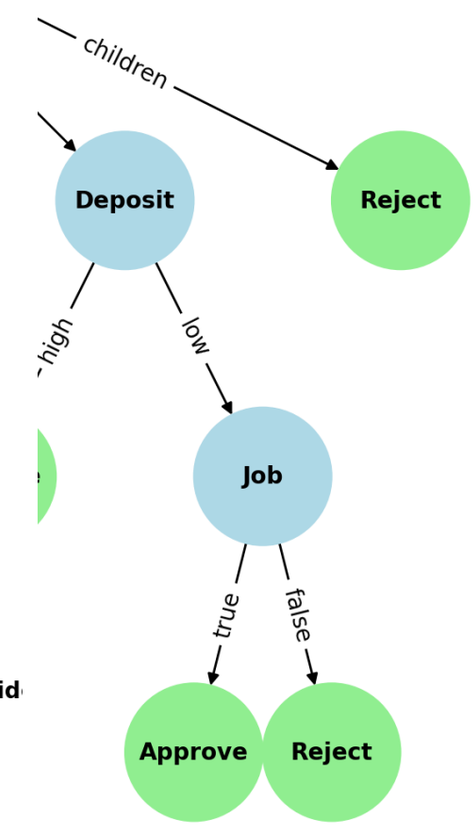
Decision Tree classifier

Decision Tree: a hierarchical, tree-structured model used for classification (and regression) tasks in machine learning

- **nodes** represent decision points
- **branches** denote possible outcomes, and
- **leaves** indicate final predictions or outcomes.

	Job	Deposit
A	true	low
B	true	low
C	true	low
D	true	high
E	false	high
1	true	low
2	false	low
3	true	low
4	false	low
5	false	high





Decision Trees vs Decision Tree Learning

- Decision Trees - Representation of the Classifier
  - Symbolic, not probabilistic
  - "Easier" to interpret compared to other models
  - Can handle both categorical and numerical data

- Decision Tree Learning - Learning Process: specifies a procedure for deciding a class, use algorithms like:
  - **ID3** (Iterative Dichotomiser 3) – Uses entropy and information gain
  - **C4.5** – Improvement over ID3, support pruning
  - **CART** (Classification and Regression Trees) – Uses Gini impurity or Mean Squared Error

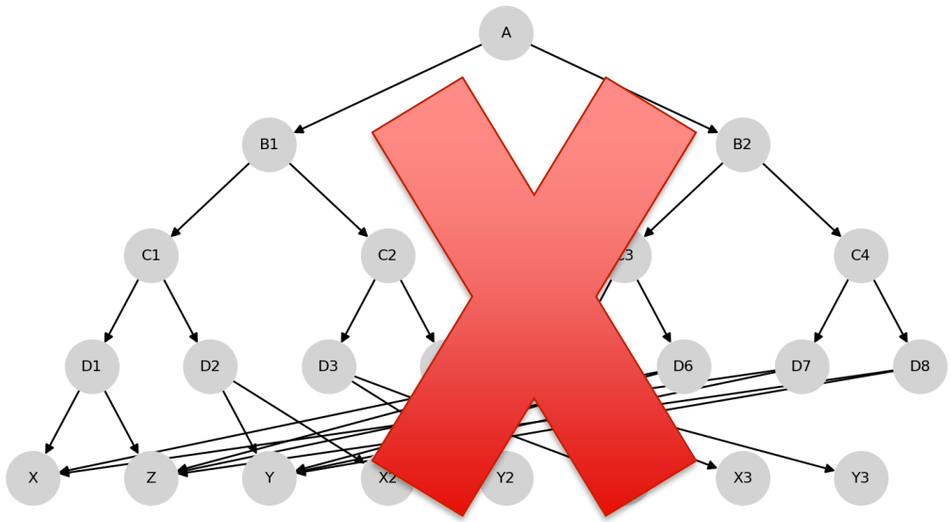
Key Differences:

- Decision tree = final model used for classification (or regression)
- Decision tree learning = process/algorithm used to construct a decision tree
- Different algorithms may lead to different tree structures

Building Decision Trees

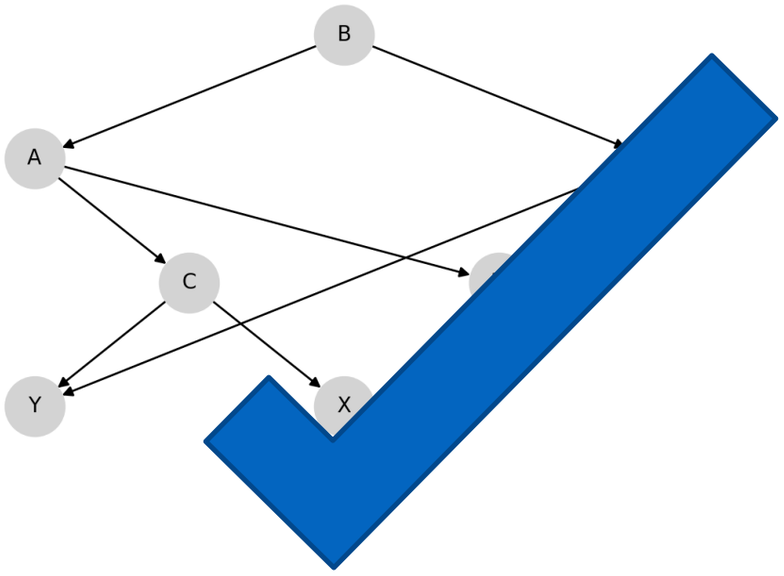
You can always build a decision tree trivially

- Choose some order on the attributes
- Build tree with one attribute for each level
- Label each leaf with appropriate class



Problems

- Each leaf represents a possible instance
- Remembering every instance
- No generalization, no prediction, no learning



Solution

- Find a small decision tree
- Capture the common characteristics of instances
- Generalize to predict classes for unseen instances

Algorithm to Learn Decision Trees

Input & Output

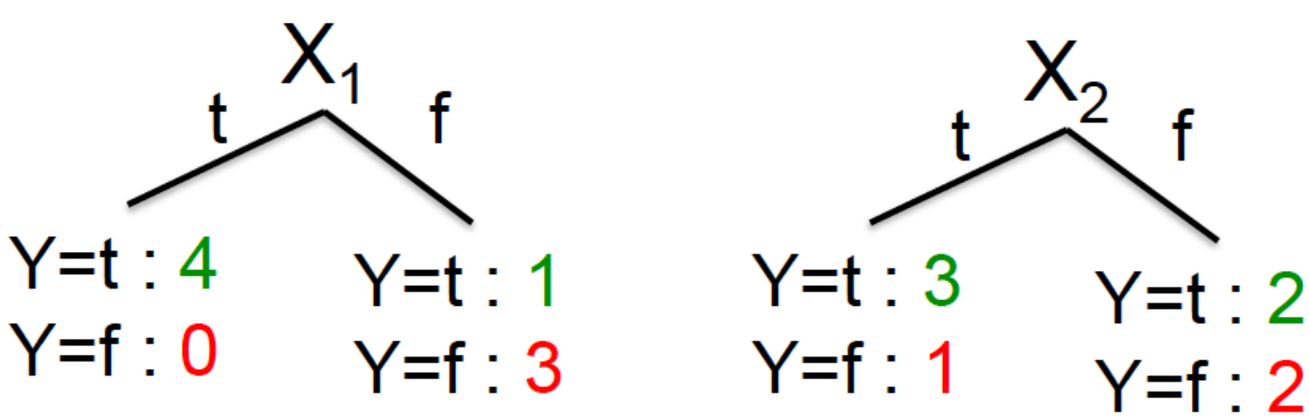
- **Input**: [Instances]
- **Output**: A "good" decision tree classifier

```
### **DT Algorithm**

┌──────────────────────────────────────────────────────────────┐
│ Examine set of instances in the root node                    │
│                                                              │
│ If set is "certain" enough, or no more attributes            │
│     then stop                                                │
│ Else                                                         │
│     Construct subsets of instances in the subnodes           │
│     Compute average "uncertainty" of subnodes                │
│     Choose the best attribute                                │
│                                                              │
│ Recurse on each subnode                                      │
└──────────────────────────────────────────────────────────────┘
```

Choose the Best Feature to Split

Should we choose x_1 or x_2 to split?

- A good feature split significantly reduce uncertainty in class labels after partitioning
- The goal is to create pure subsets
- Analyze class distributions at leaf nodes to estimate uncertainty and assess split quality.



X_1	X_2	Y
T	T	T
T	F	T
T	T	T
T	F	T
F	T	T
F	F	F
F	T	F
F	F	F

Measure Uncertainty

- Good split reduce uncertainty in class labels after split
  - Deterministic - good (all true or all false)
  - Uniform distribution- bad

- Question: What about distributions in between?

P(Y=A) = 1/2	P(Y=B) = 1/4	P(Y=C) = 1/8	P(Y=D) = 1/8

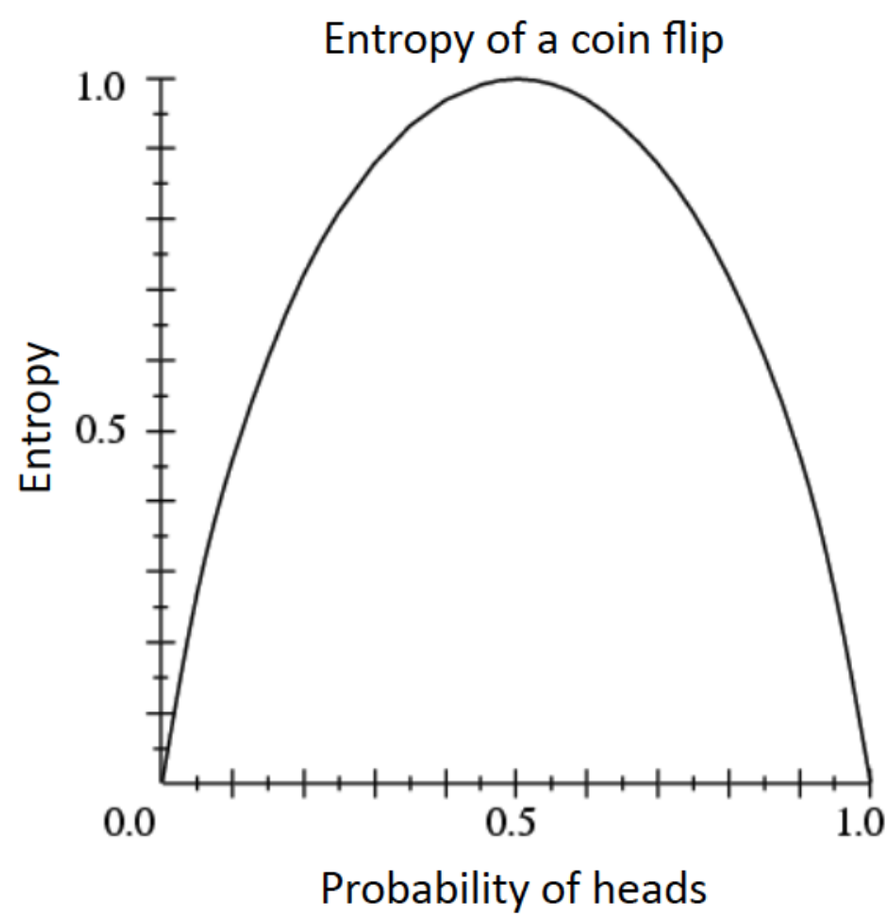
P(Y=A) = 1/4	P(Y=B) = 1/4	P(Y=C) = 1/4	P(Y=D) = 1/4

Entropy

- Entropy of a random variable Y

$$H(Y) = -\sum_{i=1}^{k} P(Y = y_i) \log_2 P(Y = y_i)$$

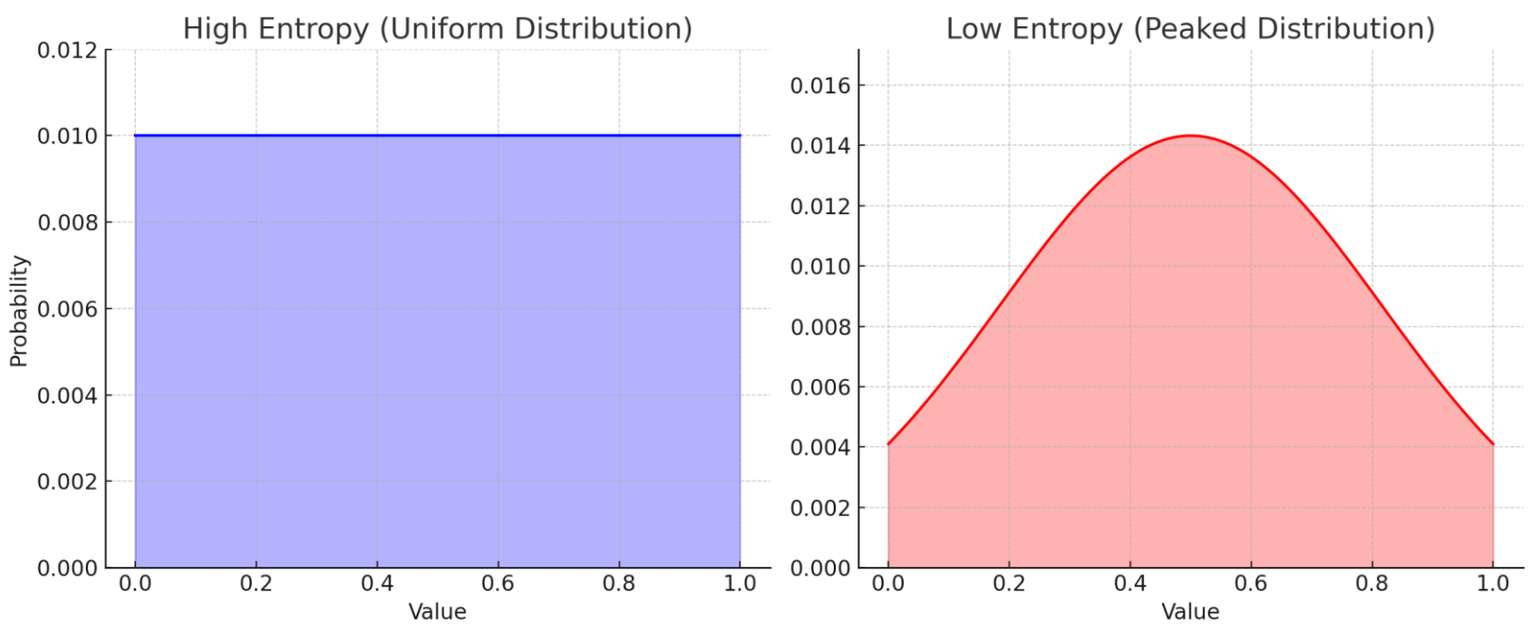
- Entropy measures the uncertainty over the value of a random variable, More uncertainty, more entropy!



High and Low Entropy

- High entropy
  - Y follows a uniform (flat) distribution
  - Values sampled from Y are less predictable

- Low entropy
  - Y follows a varied (peaks and valleys) distribution
  - Values sampled from Y are more predictable



Entropy Calculation Example

$$H(Y) = -\sum_{i=1}^{k} P(Y = y_i) \log_2 P(Y = y_i)$$

P(Y=t) = 5/6

P(Y=f) = 1/6

H(Y) = - 5/6 $\log_2$ 5/6 - 1/6 $\log_2$ 1/6

= 0.65

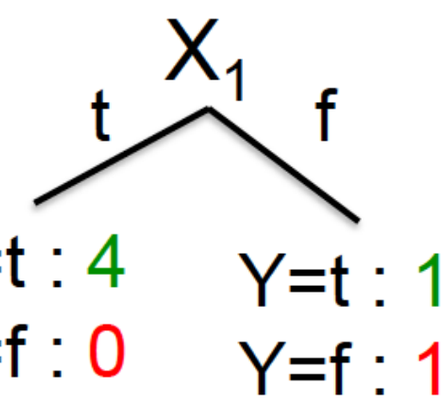
X_1	X_2	Y
T	T	T
T	F	T
T	T	T
T	F	T
F	T	T
F	F	F

Conditional Entropy

- Conditional entropy $H(Y|X)$ of a random variable Y conditioned on another random variable X

$$H(Y \mid X) = -\sum_{j=1}^{v} P(X = x_j) \sum_{i=1}^{k} P(Y = y_i \mid X = x_j) \log_2 P(Y = y_i \mid X = x_j)$$

Example:



X_1	X_2	Y
T	T	T
T	F	T
T	T	T
T	F	T
F	T	T
F	F	F

$P(X_1=t) = 4/6$

$P(X_1=f) = 2/6$

$$H(Y|X_1) = -\,4/6\,(1 \log_2 1 + 0 \log_2 0) - 2/6\,(1/2 \log_2 1/2 + 1/2 \log_2 1/2) = 2/6$$

Information Gain

- Information Gain: decrease in entropy (uncertainty) after using a feature for splitting

$$IG(X) = H(Y) - H(Y \mid X)$$

In our running example:

$IG(X_1) = H(Y) - H(Y|X_1)$
$= 0.65 - 0.33$

$IG(X_1) > 0 \rightarrow$ we prefer the split!

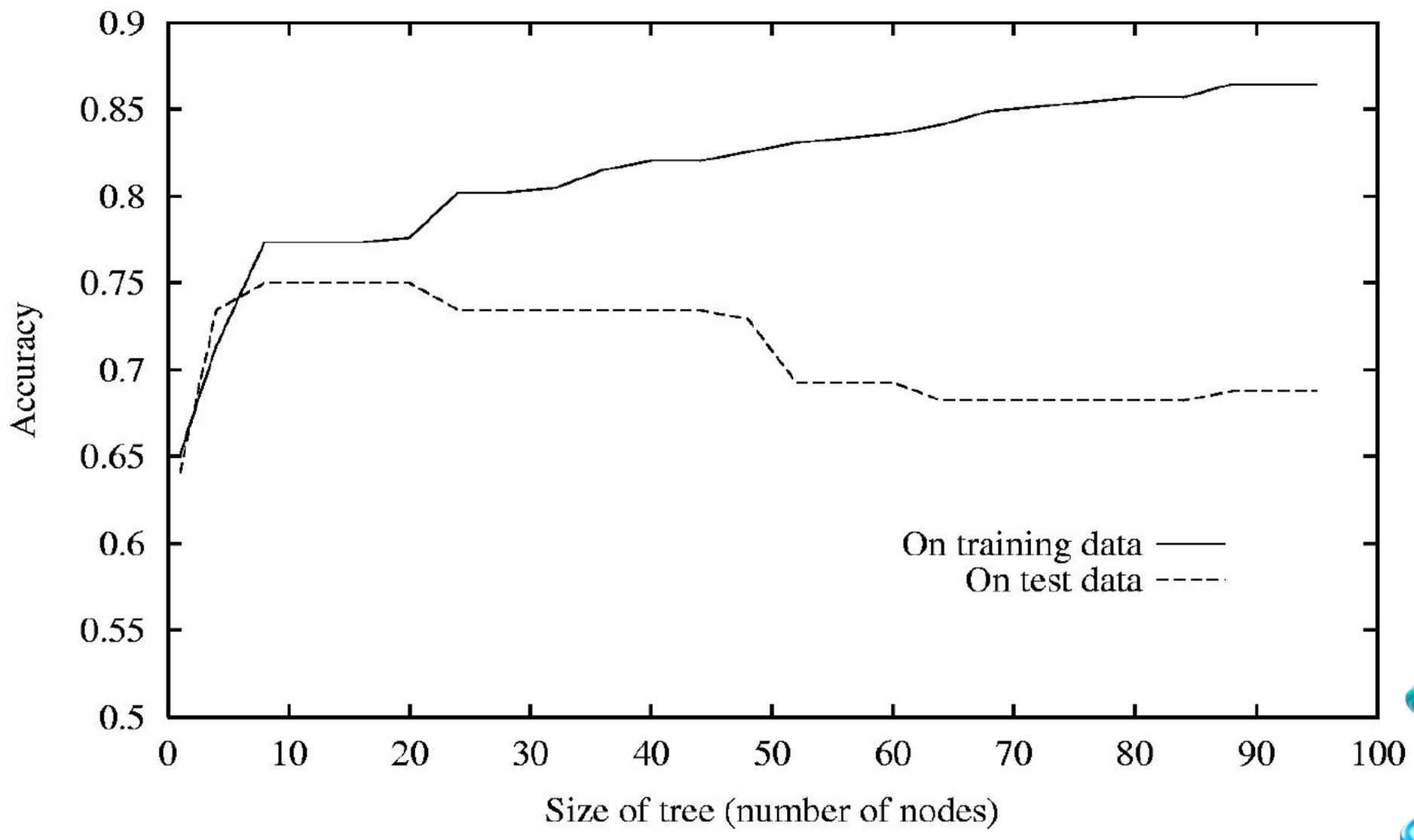
X_1	X_2	Y
T	T	T
T	F	T
T	T	T
T	F	T
F	T	T
F	F	F

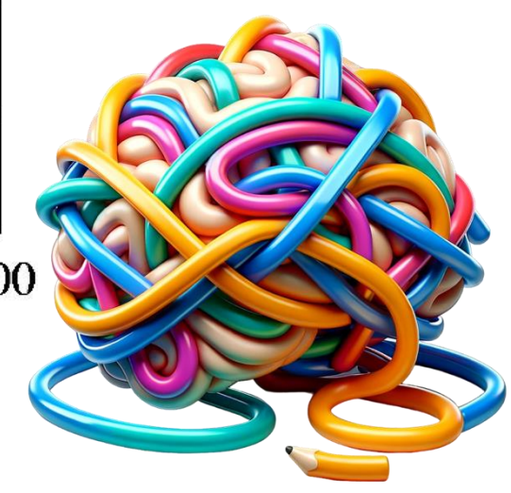
- This method is used in ID3 and C4.5 decision tree algorithms

Decision Trees can Overfit

Standard decision trees have learning bias

- The learned decision tree can be very large.
- The accuracy on the training set can be high.
- However, the accuracy on the test set can be poor





Decision Trees can Overfit

To address overfitting, we must introduce a learning bias towards simple trees

- Set the maximum tree depth
- Set the minimum partition size of each node (including the leaf node)

- Pruning: simplifying a tree
  - Pre-pruning: stop the tree from growing before perfectly classify the training data, stop when the gain is too small or number of instances is too small
  - Post-pruning: grow a full tree first and then simplify the tree, removing branches that add little value or are likely overfitting the data

Handle Real-Valued Features

- What should we do if some of the input features have real values?

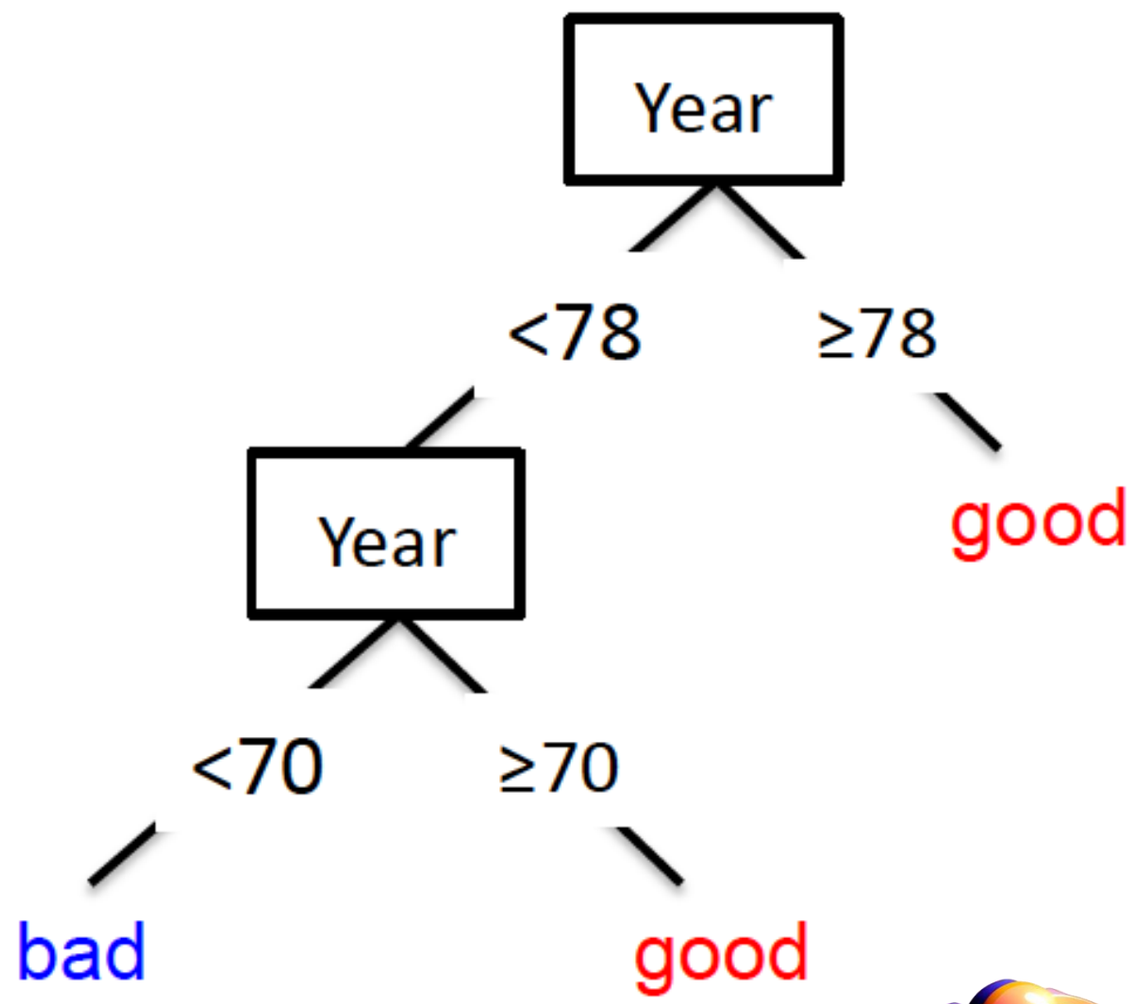
Infinite number of possible split values!!!

mpg	cylinders	displacemen	horsepower	weight	acceleration	modelyear	maker
good	4	97	75	2265	18.2	77	asia
bad	6	199	90	2648	15	70	america
bad	4	121	110	2600	12.8	77	europe
bad	8	350	175	4100	13	73	america
bad	6	198	95	3102	16.5	74	america
bad	4	108	94	2379	16.5	73	asia
bad	4	113	95	2228	14	71	asia
bad	8	302	139	3570	12.8	78	america
:	:	:	:	:	:	:	:
:	:	:	:	:	:	:	:
:	:	:	:	:	:	:	:
good	4	120	79	2625	18.6	82	america
bad	8	455	225	4425	10	70	america
good	4	107	86	2464	15.5	76	europe
bad	5	131	103	2830	15.9	78	europe

Threshold based Split

- Binary tree: split on any feature x based on threshold t
  - Branch 1: $x < t$
  - Branch 2: $x \geq t$

- Small change of the learning algorithm is required
  - Allow repeated split of the same input feature based on different thresholds



Threshold based Split

- Don’t need to try each possible split point - Order items and only consider class boundaries!
- Consider the class boundaries, choose the best split

Job	Deposit	Family	Class
1	$4K	couple	Reject
4	$6K	single	Reject
B	$7K	couple	Approve
5	$8K	children	Reject
A	$10K	single	Approve
E	$12K	couple	Approve
3	$15K	children	Reject
C	$16K	single	Approve
D	$25K	single	Approve
2	$30K	couple	Reject

- Deposit < 7K : IG = 0.188
- Deposit < 8K. : IG = 0.238
- Deposit < 10K: IG = 0.208
- Deposit < 15K: IG = 0.250
- Deposit < 16K: IG = 0.238
- Deposit < 30K: IG = 0.222

Which one should we choose?

- Best Split: Deposit < 15K with the highest IG

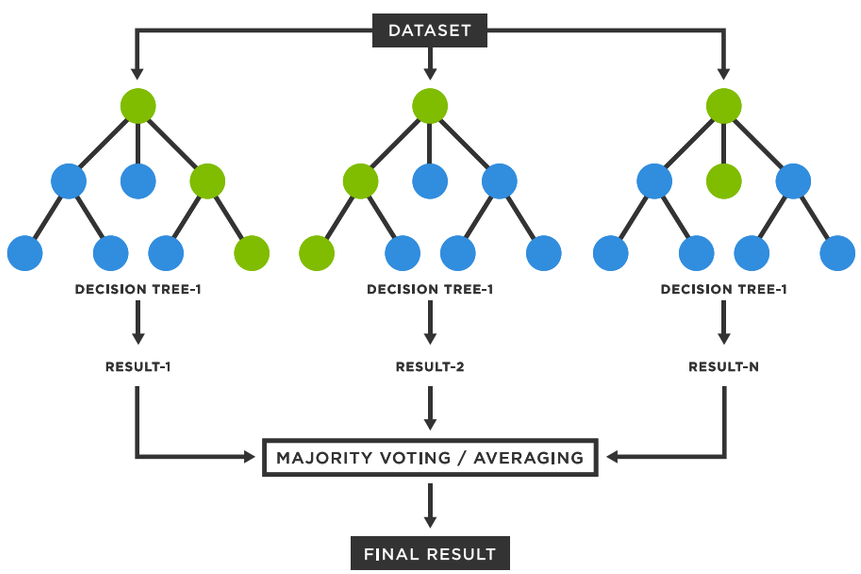
Example – DT in Scikit-learn

```python
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score

# Sample dataset (Iris dataset)
from sklearn.datasets import load_iris
iris = load_iris()
X, y = iris.data, iris.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create a Decision Tree classifier with Information Gain (Entropy) as the criterion
clf = DecisionTreeClassifier(criterion="entropy", random_state=42)

# Train the classifier
clf.fit(X_train, y_train)

# Make predictions
y_pred = clf.predict(X_test)

# Evaluate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
```

Random Forests

- Random Forest: an ensemble learning method that builds multiple decision trees and merges their outputs to improve accuracy and reduce overfitting
  - Each tree is built from a bootstrap sample (i.e. a sample drawn with replacement) from the training set
  - Each split is found either from all input features or a random subset of features
  - The final prediction is based on aggregating results from all trees with majority voting

Tree 1	Tree 2	Tree 3	Tree 4	Tree 5	Final Prediction
Spam	Not Spam	Spam	Spam	Not Spam	Spam (Majority)



https://www.tibco.com/reference-center/what-is-a-random-forest

Example - Random Forests with Scikit-learn

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.tree import plot_tree

# Load the Iris dataset
iris = datasets.load_iris()
X, y = iris.data, iris.target

# Split dataset into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create a Random Forest classifier
clf = RandomForestClassifier(n_estimators=100, criterion="gini", max_depth=3, random_state=42)

# Train the model
clf.fit(X_train, y_train)

# Make predictions on the test set
y_pred = clf.predict(X_test)

# Compute accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
```

Supervised Learning for Classification

- So far …
  - K-nearest neighbors (KNN)
  - SVM
  - Decision Trees
  - Random Forests

- Which technique should we pick?
  - For interpretability → Decision Trees
  - For high accuracy → Random Forest or SVM
  - For large datasets → Random Forest (parallelizable)
  - For non-linear data → SVM with kernels
  - For quick classification → KNN (if dataset is small)